

Reviewer Pack — Minimal Rank of Matroid Invariant under Expansion for Monoto...

Entry 9ad49bd886a7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 05:08:40 UTC

Minimal Rank of Matroid Invariant under Expansion for Monotone k-CLIQUE

Entry ID: 9ad49bd886a7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 05:08:40 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Monotone Circuit Complexity

Statement:

[‘For any monotone DNF formula representing the k-CLIQUE problem, the rank of its associated matroid is at least $\Omega(n^{(k/2)})$ under a constructive mapping.’, ‘A falsifier-friendly formulation: There exists a monotone DNF formula for k-CLIQUE with a matroid rank bounded by $O(n^\beta)$ where $\beta < (k/2)$ and this is not the case for all k-CLIQUE instances.’, ‘The constructive mapping involves representing each clause as an element in a vector space over a finite field, forming an incidence matrix, and associating the resulting matroid to the formula.’]

Rationale (proposer’s reasoning):

[‘Matroid theory provides a framework for studying the structure of sets with binary relationships. By examining the matroid associated with a monotone DNF formula, we can potentially gain insights into the complexity of solving the k-CLIQUE problem.’, ‘This conjecture bridges the gap between abstract algebraic structures and computational complexity, aiming to expose new invariants that could lead to improved understanding or algorithms for solving k-CLIQUE.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d8c33468f97adc5b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a monotone DNF formula representing k-CLIQUE, if the associated matroid’s rank exceeds or equals $n^{(k/2)}$ for all seeds and the correlation coefficient between the matroid rank and circuit size is greater than 0.5, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "matroid theory" AND "monotone k-CLIQUE" - "rank of associated matroid" AND monotone DNF formula AND k-CLIQUE - "constructive mapping" in vector space AND finite field AND monotone DNF formula

Top relevant hits considered: - [http://arxiv.org/abs/2412.05161v1] DNF: Unconditional 4D Generation with Dictionary-based Neural Fields - [http://arxiv.org/abs/0704.1574v3] Retarded electric and magnetic fields of a moving charge: Feynman's derivation of Liénard-Wiechert potentials revisited - [http://arxiv.org/abs/2208.13728v1] Compressible Topological Vector Spaces

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def mod_inverse(a, m):
    gcd, x, _ = extended_gcd(a, m)
    if gcd != 1:
        raise ValueError(f"No modular inverse for {a} modulo {m}")
    return x % m
```

```

def gaussian_elimination(A, b):
    n = len(b)
    A_augmented = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(A_augmented[r][i]))
        A_augmented[i], A_augmented[max_row] = A_augmented[max_row], A_augmented[i]
        factor = A_augmented[i][i]
        for j in range(i, n + 1):
            A_augmented[i][j] /= factor
        for k in range(n):
            if k != i:
                factor = A_augmented[k][i]
                for j in range(i, n + 1):
                    A_augmented[k][j] -= factor * A_augmented[i][j]
    return [row[-1] for row in A_augmented]

def dnf_to_matrix(dnf):
    n = len(dnf)
    m = max(len(clause) for clause in dnf)
    matrix = [[0] * m for _ in range(n)]
    for i, clause in enumerate(dnf):
        for literal in clause:
            if literal > 0:
                matrix[i][literal - 1] = 1
            else:
                matrix[i][-literal - 1] = 1
    return matrix

def rank(matrix):
    n, m = len(matrix), len(matrix[0])
    A = [row[:] for row in matrix]
    r = 0
    for j in range(m):
        i_max = next((i for i in range(r, n) if A[i][j]), None)
        if i_max is not None:
            A[r], A[i_max] = A[i_max], A[r]
            for i in range(r + 1, n):
                factor = A[i][j] / A[r][j]
                for k in range(m):
                    A[i][k] -= factor * A[r][k]
            r += 1
    return r

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_dnf(n, k):
        clauses = []
        for _ in range(k):
            clause = set()
            while len(clause) < n:
                literal = random.randint(1, 2 * n)
                if literal > n and -literal not in clause:
                    clause.add(-literal)
                elif literal <= n and literal not in clause:

```

```

        clause.add(literal)
        clauses.append(list(clause))
    return clauses

def circuit_size(dnf):
    n = len(dnf)
    m = max(len(clause) for clause in dnf)
    matrix = dnf_to_matrix(dnf)
    rank_val = rank(matrix)
    return n - rank_val + 1

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    dnf = generate_dnf(n, n // 2)
    matroid_rank = rank(dnf_to_matrix(dnf))
    circuit_size_val = circuit_size(dnf)
    results.append({
        "n": n,
        "matroid_rank": matroid_rank,
        "circuit_size": circuit_size_val
    })

if not results:
    return {
        "metric_name": "Matroid Rank vs Circuit Size",
        "metric_value": 0.0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No instances generated"
    }

matroid_ranks = [res["matroid_rank"] for res in results]
circuit_sizes = [res["circuit_size"] for res in results]

mean_matroid_rank = sum(matroid_ranks) / len(matroid_ranks)
std_dev_matroid_rank = math.sqrt(sum((x - mean_matroid_rank) ** 2 for x in matroid_ranks) / len(matroid_ranks))
correlation_coefficient = sum((matroid_ranks[i] - mean_matroid_rank) * (circuit_sizes[i] - sum(circuit_sizes) / len(circuit_sizes))) / (len(matroid_ranks) * std_dev_matroid_rank * std_dev_circuit_size)

conjecture_holds = correlation_coefficient > 0.5
counterexample = "" if conjecture_holds else "Correlation coefficient below threshold"

return {
    "metric_name": "Matroid Rank vs Circuit Size",
    "metric_value": mean_matroid_rank,
    "instances_tested": len(matroid_ranks),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []

```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, **result}}")
    results.append(result)

mean_metric_value = sum(res["metric_value"] for res in results) / len(results)
std_dev_metric_value = math.sqrt(sum((res["metric_value"] - mean_metric_value) ** 2 for res in results) / len(results))
support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

if all(res["conjecture_holds"] for res in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_dev_metric_value} support_fraction={support_fraction}")
elif any(not res["conjecture_holds"] for res in results):
    first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient below threshold\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE Reason=No instances generated")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b5b29a8.py", line 153, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b5b29a8.py", line 111, in run_trial
    matroid_rank = rank(dnf_to_matrix(dnf))
                    ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6b5b29a8.py", line 65, in dnf_to_matrix
    matrix[i][-literal - 1] = 1
    ~~~~~~^~~~~~
IndexError: list assignment index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate the cause of the crash and attempt to run the test again.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17839
2	propose	ollama_remote	glm4:latest	0	0	19837
3	preregistration	ollama_remote	glm4:latest	0	0	5661
4	novelty	ollama_remote	glm4:latest	0	0	4706
5	novelty	ollama_remote	glm4:latest	0	0	6013
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35270
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11278
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10531
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18367
10	judge	ollama_remote	glm4:latest	0	0	7882

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 137385 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/9ad49bd886a7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9ad49bd886a7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9ad49bd886a7.tar.gz` (if generated)